LOSSLESS DATA COMPRESSION TOOL

TAI PROJECT #1 - 2025/26

Guilherme Rosa
Student ID: 113968
Universidade de Aveiro
Aveiro, Portugal
guilherme.rosa@ua.pt

João Roldão
Student ID: 113920
Universidade de Aveiro
Aveiro, Portugal
joao.roldao@ua.pt

Henrique Teixeira
Student ID: 114588
Universidade de Aveiro
Aveiro, Portugal
henrique.teixeira@ua.pt

April 13, 2026

ABSTRACT

We developed and benchmarked a lossless compression tool for the TAI course project, centered on statistical coding methods based on range-coder-style entropy coding. The repository now exposes three distinct design points. `v9` is the strongest compression-oriented project variant in the current benchmark snapshot, reaching a 54.40% average compressed size on the 8-file corpus (13 MiB total). `v8` is the fastest project variant in average compression and decompression time, using an LZ77-style token stream without entropy coding and averaging 26 ms compression and 24 ms decompression. Even so, the most balanced overall design is `v5`: it combines BWT, MTF, optional zero-run encoding, and an adaptive order-1 model with range coding/rANS backends, achieving 54.77% while remaining far faster to compress than `v9` and much more compact than `v8`. The resulting comparison highlights the main project trade-off between compression strength, throughput, and overall practical balance.

1 Introduction

Lossless compression is fundamentally a two-part problem: the encoder must first model the input so that probable symbols receive high estimated probability, and only then can the coder map those probabilities into short bit representations. In practice, the quality of a compressor depends on how well the model captures structure in the data and how efficiently the coding stage turns that structure into a compact stream.

This project was developed in the context of the Algorithmic Information Theory course and evolved through several compression strategies. The repository now contains three especially relevant variants. `v5` is the clearest balanced milestone and still offers the best overall speed/ratio balance; `v9` extends the same statistical family and achieves the best compression ratio among the project versions; and `v8` explores the opposite design point, replacing the statistical pipeline with a very fast LZ77-style matcher.

The goal of this report is therefore to explain the main design choices, summarize the implementation strategy, and present the measured trade-offs on the benchmark corpus. In particular, we use `v5` as the main case study for the project's best overall compromise, `v9` as the ratio leader, and `v8` as the speed-oriented contrast.

2 Background

This project is about lossless compression, so the original file must be recovered exactly after decompression. The main idea used in our work is simple: first reorganize the data so patterns become easier to see, then model the data, and finally encode it into fewer bits.

Our main compression pipeline follows this logic. In the statistical versions, the data can first pass through the Burrows–Wheeler Transform (BWT) and Move-to-Front (MTF) [1]. These steps do not compress the file by themselves,

but they help group similar values together, which makes the next stage more effective. After that, the compressor uses an adaptive model and range coding to generate the final compressed output [2, 3].

Not all versions of the project use the same approach. The more balanced versions, especially `v5` and `v9`, follow the statistical pipeline described above. In contrast, `v8` uses a simpler LZ77-style method [4, 5]. It is much faster, but it usually produces larger files. This comparison is useful because it shows the main trade-off explored in the project: stronger compression usually needs more processing time.

3 Methodology

3.1 System Architecture

The report uses a version-focused methodology. Rather than treating the repository as a single static compressor, we identify three representative designs. `v5` is the main balanced design, combining classic modeling/coding separation with practical runtime. `v9` is the stronger compression-oriented extension of the same family. `v8` is the speed-focused contrast that removes the statistical pipeline and uses direct dictionary matching. This framing exposes the central trade-off between ratio, throughput, and overall balance.

3.2 Algorithm Selection

The `v5` line remains the primary subject because it offers the best overall speed/ratio trade-off in the refreshed benchmark set and best illustrates the repository's statistical compression pipeline. BWT, MTF, and zero-run coding reshape low-entropy structure before modeling; order-1 adaptive modeling captures local byte dependencies better than a static order-0 model; and range coding/rANS provide the coding stage while keeping the implementation efficient. `v9` is included because it improves ratio further through per-block candidate selection and broader transform choices, while `v8` is kept as a comparison point because its strength is speed, not compactness.

3.3 Optimization Strategy

Development proceeded iteratively. Earlier versions established the basic order-0 and range-coding path, then introduced BWT preprocessing and faster data structures. The `v5` line consolidated the strongest ideas into one pipeline: block-based BWT, MTF, optional zero-run encoding, adaptive order-1 modeling, and parallel block processing. `v8` then explored the opposite extreme by discarding preprocessing and entropy coding entirely in order to maximize speed. Finally, `v9` extended the statistical line with per-block candidate search over multiple transforms and model orders to push ratio further.

3.4 Benchmarking

All claims in this report are based on the benchmark corpus used throughout the project. The corpus contains eight files (A–H) totaling 13 MiB, with different characteristics including text, structured binaries, high-entropy data, and an incompressible random case. The reported metrics are compressed size, compression ratio, bits per symbol, compression time, decompression time, and lossless correctness. The benchmarks compare `v5`, `v8`, and `v9` against gzip, bzip2, xz, and zstd. Benchmarks used the standard release configuration from the project Makefile (g++/C++17 with `-O3 -march=native -flto=auto`) on the same 8-file, 13 MiB corpus.

4 Implementation

4.1 BWT-Based Statistical Line

The main statistical line in the repository is block-oriented. In `v5`, structured data is passed through BWT so that symbols occurring in similar contexts are grouped together. This creates the distributional skew later exploited by MTF and entropy coding. The implementation history shows that larger and better-balanced blocks improved compression because they preserved more context across each transform pass.

4.2 Move-to-Front and Run-Length Encoding

After BWT, the transformed block is passed through MTF. This turns repeated local symbol clusters into many low ranks, especially zeros. A zero-run stage is then optionally applied to shrink long zero runs further. The value of this

preprocessing chain is practical rather than theoretical: it reshapes the byte stream into a form that a simple context model can code effectively. The repository notes one important limitation here, namely the handling of rank 255 in the zero-run stage, which helps explain why some binary-heavy files remain difficult.

4.3 Context Models and Coders

The statistical core of `v5` is an adaptive order-1 model implemented over flat arrays instead of pointer-heavy dynamic structures. This keeps the model compact and cache-friendly. The design also incorporates a PPM-style escape path: when the current order-1 context cannot emit a symbol directly, the coder escapes to a lower-order distribution. Two refinements described in the repository are especially relevant: Method C exclusions, which remove already-seen symbols from the fallback distribution, and singleton-based escape estimation, which makes escape probabilities depend on how many symbols are still weakly supported in the current context.

The compliant coding stage is based on range coding, with rANS used in some static order-0 cases as a faster alternative backend. In architectural terms, `v5` remains a modeling-first compressor: preprocessing and prediction define a probability model, and the coder converts that model into the final bitstream.

4.4 From v5 to v9

The main architectural change in `v9` is that it no longer commits to one preprocessing path for the entire file. Instead, it performs parallel per-block candidate search over raw, BWT-based, LZP-prefiltered, and x86-filtered variants, and it can choose between order-1 and order-2 statistical models. The selected block configuration is stored with explicit parallel-header metadata and transform flags, allowing the decompressor to replay the exact sequence of transforms. This broader search explains why `v9` improves ratio over `v5`, but it also explains the much larger compression cost.

4.5 Speed-Oriented Variant

By contrast, `v8` follows a much simpler implementation strategy. It uses a single-pass LZ77-style matcher with a small hash table, byte-aligned tokens, and no entropy coder. This makes the code much smaller and much faster, but it also removes the stronger statistical modeling that makes `v5` and `v9` competitive on compression ratio.

5 Architecture Diagrams

This section summarizes the compression-side architecture of the three project variants with simple left-to-right diagrams. The goal is to show the main pipeline differences between the balanced statistical design of `v5`, the speed-oriented LZ77 path of `v8`, and the broader per-block search used by `v9`.

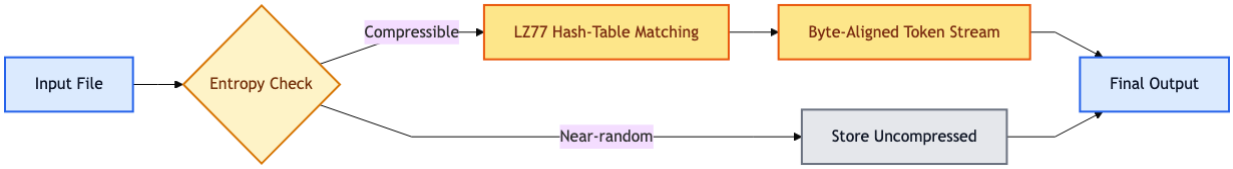
5.1 `v5`

`v5` follows a fixed statistical pipeline: it splits the input into blocks, reshapes each block with BWT and MTF, optionally shrinks zero runs, and then applies adaptive modeling before entropy coding.

High-level compression pipeline of `v5`.

5.2 `v8`

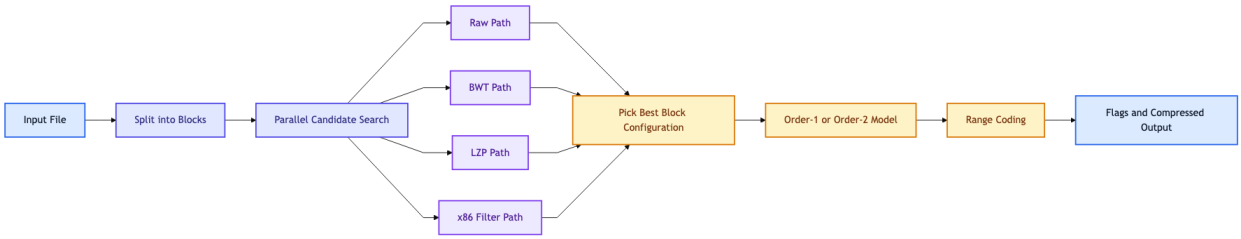
`v8` removes the transform and entropy-coding stages entirely. Instead, it relies on a fast LZ77-style matcher with a simple fallback that stores near-random data uncompressed.



High-level compression pipeline of `v8`.

5.3 v9

v9 extends the statistical line by testing several block configurations in parallel. After that search, it stores the chosen transform path and model so that decompression can replay the same sequence.



High-level compression pipeline of v9.

6 Results

6.1 Compression Ratio Evolution

The repository history shows a clear progression from a simple order-0 baseline toward stronger BWT-based statistical compressors. The key milestones for the purposes of this report are v5, which consolidated the best balanced requirement-compliant implementation; v8, which optimized for speed rather than ratio; and v9, which pushed the statistical line further and became the strongest project variant in compression ratio.

6.2 Benchmark Comparison

Table 1: Aggregate benchmark results from the 8-file repository corpus. Lower ratio and bits/symbol are better.

Tool	Comp. bytes	Avg. ratio	Bits/symbol	Avg. comp. time	Avg. decomp. time
v5	7,194,380	54.77%	4.3814	98 ms	101 ms
v8	10,038,825	76.42%	6.1136	**26 ms**	**24 ms**
v9	7,145,565	**54.40%**	4.3516	789 ms	126 ms
gzip	7,814,799	59.49%	4.7592	111 ms	22 ms
bzip2	7,209,293	54.88%	4.3905	177 ms	90 ms
xz	7,114,088	54.16%	4.3325	456 ms	65 ms
zstd	7,695,793	58.58%	4.6867	20 ms	13 ms

Table 1 shows three distinct optima. v9 is the strongest project version in compression ratio at 54.40%, improving on v5's 54.77% and slightly beating bzip2's 54.88%, although it still remains behind xz at 54.16%. v8 is the clear speed winner among the project versions. Even so, v5 remains the most balanced overall design: its ratio stays close to v9, while its compression time is far lower and its output quality is vastly better than v8.

The file-level results show a more nuanced picture. v9 wins files A, E, and G; v5 wins B and D; xz wins C and H; and bzip2 wins F. Against bzip2 specifically, v9 produces smaller outputs on A, B, C, D, E, and G, but loses on F and H. The gains are most convincing on structured inputs such as A and G, where the broader per-block search of v9 can choose a better transform/model combination than the fixed v5 pipeline.

The contrast with v8 is even sharper. On file B, the ratio is 17.35% for both v5 and v9, but 37.84% for v8; on file G, the values are 28.82%, 28.48%, and 72.76%; and on file H they are 43.79%, 43.64%, and 58.02%. In other words, v8 demonstrates that much of the runtime cost in the statistical variants comes from modeling and coding sophistication rather than from file I/O or framework overhead.

From an information-theoretic viewpoint, file D behaves essentially as an incompressible source: all tools remain at about 8 bits/symbol, so the main objective there is minimal overhead rather than real compression. Files F and H instead expose model mismatch. F is only weakly compressible and fairly binary-heavy, so the current statistical line captures less exploitable structure than bzip2, which explains why bzip2 reaches 81.06% while v5 and v9 remain at 82.42% and 81.70%. H is clearly compressible, but xz reduces it to 2.87 bits/symbol whereas the project variants stay near 3.5 bits/symbol, indicating that substantial longer-range structure remains uncaptured by the present models.

6.3 Performance Analysis

From a systems perspective, the repository exposes three practical conclusions. First, stronger modeling is worthwhile on structured inputs because the ratio improvements are real, but the extra search cost can become large. Second, parallel block processing is important for making a BWT-based pipeline practical. Third, once entropy coding and preprocessing are removed, throughput increases dramatically, but the output size deteriorates so much that the result is better seen as a different product goal rather than a direct replacement. This is why `v5` still stands out as the best overall balance, even though `v9` compresses slightly better and `v8` runs much faster.

7 Conclusion

This project produced a credible lossless compression tool and a clear comparison between three different design choices. In the final benchmark set, `v9` achieved the best compression ratio, `v8` was the fastest version, and `v5` remained the best overall balance. It combined good compression performance with much lower cost than `v9` and much better output size than `v8`.

A main lesson from the project is that compression performance did not come from one isolated algorithmic idea, but from how preprocessing, modeling, and implementation details worked together. BWT and MTF were useful because they made the statistical model more effective, while choices such as block handling, escape estimation, and candidate selection also had a strong impact. The comparison with `v8` also made the trade-off very clear: higher speed is possible, but usually at the cost of weaker compression.

Future work can follow the same two directions already suggested by the repository. One direction is to keep improving the statistical line by reducing the cost of broader search and handling difficult binary files better. The other is to continue the faster `v8` direction as a separate compressor for cases where throughput matters more than compactness.

References

[1] Michael Burrows and David J Wheeler. A block-sorting lossless data compression algorithm. In *Digital SRC Research Report*, 1994.

[2] Jorma Rissanen and Glen G Langdon. Arithmetic coding. *IBM Journal of research and development*, 23(2):149–162, 1979.

[3] Michael Schindler. A fast renormalisation for arithmetic coding. Technical report, 1998.

[4] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977.

[5] Yann Collet. Lz4: Extremely fast lossless compression algorithm. `https://lz4.org/`. Accessed 2026-04-11.